

Pillar 1: Sales Forecasting – Task Completion Report

Adam Aberbach

30 March 2026

Task Completion Report

Author: Adam Aberbach

Pillar: 1 – Sales Forecasting

Date: Monday, 30 March 2026

SRS Version: 2.0

Coordinator: Reda

This report documents the tasks completed for the Sales Forecasting pillar of the Loving Loyalty AI Intelligence Suite. Where tasks required coordination with Yahya Hammoudeh, the collaboration is noted. The verification findings from Report B (Verification and Audit of Forecasting Model Optimization, March 2026) informed several of the fixes and design decisions described below.

1. Expanding Mean Leakage Fix

SRS Reference: Section 7, Adam Task 1

Report B Reference: Section 4 (Data Leakage – PASS with minor caveat)

Deliverables: Fixed `autoresearch/evaluate.py`, fixed `src/features/lag_features.py`, new `src/forecast/feature_engineer.py`

Report B's independent audit identified that the `expanding_mean` feature was computed across the entire dataset before the train/test split. This meant test-period rows incorporated demand from later test days into their expanding mean, constituting target leakage.

I fixed this in three locations:

autoresearch/evaluate.py: Replaced the leaky computation with a shifted version that only looks at past values:

```
# Before (leaky):
df["expanding_mean"] = df.groupby(grp)["quantity_sold"].expanding().mean()

# After (fixed):
df["expanding_mean"] = df.groupby(grp)["quantity_sold"].transform(
```

```
lambda x: x.shift(1).expanding(min_periods=1).mean()
)
```

src/features/lag_features.py: Added documentation referencing Report B and clarifying that callers must pass only the training partition to prevent leakage.

src/forecast/feature_engineer.py: The new modularized feature engineering module (Task 2) implements `expanding_mean` with a `fit/transform` interface. The `fit()` method computes `expanding_mean` on training data only and freezes the last known value per `(place_id, item_id)` pair. The `transform()` method applies the frozen values to any new data, ensuring no future information leaks into inference.

The autoresearch cache file (`autoresearch/.cache/prepared_data.pkl`) was deleted to force regeneration with the corrected feature.

Collaboration with Yahya: Yahya's validation report (`docs/validation/real_data_validation`) references this fix and notes that the post-fix `expanding_mean` no longer triggers the Report B caveat. The audit finding now reads PASS without caveats.

2. Codebase Modularization

SRS Reference: Section 7, Adam Task 2

Deliverables: `src/forecast/data_loader.py`, `src/forecast/feature_engineer.py`, `src/forecast/trainer.py`, `src/forecast/predictor.py`, `src/forecast/__init__.py`

I modularized the forecasting pipeline into four production-ready Python modules, each with a clear single responsibility:

data_loader.py

Handles data ingestion from MySQL (production) or CSV (development). Implements:

- UNIX timestamp to datetime conversion
- `WHERE status = 'Settled'` filter on payment data as specified in SRS Section 3
- Daily demand aggregation per `(place_id, item_id)` pair
- Complete date grid creation with zero-filling for missing days
- Automatic fallback from MySQL to CSV when database is unavailable

feature_engineer.py

Full feature engineering pipeline with a scikit-learn-compatible `fit/transform` interface:

- **Time features:** `day_of_week`, `is_weekend`, `month`, `week_of_year`, `is_public_holiday`, `days_to_next_holiday`
- **Lag features:** 1d, 7d, 14d, 28d with proper `shift(1)` to prevent leakage
- **Rolling statistics:** `rolling_mean_7d`, `rolling_mean_14d`, `rolling_std_7d`, `rolling_max_7d`

- **Trend features:** wow_growth_rate, recent_vs_expanding_mean
- **Seasonality:** Fourier order 2 (sin/cos for weekly and annual cycles)
- **Interaction features:** lag7d x day_of_week, rolling_mean x is_weekend
- **expanding_mean:** Computed only on training data with frozen values for inference (the leakage fix from Task 1)

All features match the specifications in SRS Section 4.

trainer.py

Trains the AdaptiveBlendModel architecture validated by the 71-experiment optimization:

- **Global model:** RandomForest (300 trees, min_samples_leaf=2) trained on all stores
- **Per-store models:** ExtraTrees (800 trees) per store, falling back to global for stores with fewer than 10 training samples
- **Blend:** 75% global + 25% per-store, adaptive by store data volume using the formula:
 - $\text{frac} = \min(\text{n_store} / \text{min_store_samples}, 1.0)$
 - $\text{global_weight} = 1.0 - \text{frac} * (1.0 - 0.75)$
 - Per-store weight ranges from 0% (fewer than 10 samples) to 25% (100+ samples)
- **Exponential decay:** half-life = 12 days
- **Newsvendor safety buffer** computed per item:
 - Critical ratio: $\text{stockout_cost} / (\text{stockout_cost} + \text{waste_cost}) = 1.5 / 1.8 = 0.833$
 - Buffer multiplier: $1 + z_{0.833} * \text{sigma} / \text{mean_demand}$
 - Stable items (low sigma): approximately 1.12 buffer
 - Volatile items (high sigma): approximately 1.45 buffer
 - Items with fewer than 30 days history: fallback to uniform 1.27 buffer
 - Buffer always clamped between 1.0 and 2.0 as required by SRS Section 5
- **Three output variants:** balanced (buffer as-is), waste_optimized (buffer x 0.90), stockout_optimized (buffer x 1.06)
- Model artifact save/load with joblib

predictor.py

Production inference module:

- Loads trained model artifacts
- Generates predictions for any date range
- Applies per-item newsvendor buffer based on variant
- Computes confidence scores per SRS Table 25:
 - 90+ days of history: 0.75 to 0.95
 - 30 to 90 days: 0.50 to 0.74
 - Fewer than 30 days: 0.20 to 0.49
 - Zero orders: 0.10 to 0.25
- Generates forecast drivers from feature importance (always at least 3, never empty)

- Returns predictions with lower and upper bounds (lower always less than predicted, upper always greater)

Collaboration with Yahya: The predictor module is used by both Yahya's POST /ai/forecast endpoint and my POST /ai/forecast/items endpoint. We coordinated to ensure the confidence scoring logic, newsvendor buffer computation, and driver generation are shared through this module rather than duplicated across endpoints.

3. Item-Level Forecast Endpoint

SRS Reference: Section 7, Adam Task 3

Deliverable: src/api/forecast_items_routes.py

I built the POST /ai/forecast/items endpoint matching the SRS Section 6 contract:

Request: - placeId (string) - merchant identifier - startDate (YYYY-MM-DD) - forecast start date - endDate (YYYY-MM-DD) - forecast end date - granularity (day | week | month | hour)

Response: - placeId (string) - itemForecasts (array), each containing: - itemTitle (string) - human-readable item name - predictions (array), each with: - date (YYYY-MM-DD) - predictedUnits (integer) - demand forecast in whole units - confidence (float, 0.0 to 1.0)

The endpoint uses the predictor module from Task 2, includes Pydantic request/response validation, proper error handling, and async thread offloading for the compute-heavy prediction step. It was registered in src/main.py alongside Yahya's forecast routes.

Collaboration with Yahya: Both endpoints share the same FastAPI application, model artifacts, and predictor module. Yahya handled the aggregate revenue forecast (POST /ai/forecast) while I handled the item-level demand forecast (POST /ai/forecast/items). Both endpoints follow the same confidence scoring rules and newsvendor buffer logic.

4. README Update

SRS Reference: Section 7, Adam Task 4

Deliverable: README.md

I created a comprehensive README for the forecasting service covering:

- **Project overview:** Loving Loyalty AI Intelligence Suite, Pillar 1 – Sales Forecasting
- **Architecture:** Adaptive blend model (RF global + ET per-store, 75/25, decay weighting)
- **Output variants with clear explanations:**

- *balanced* - Default. Applies the newsvendor-optimal safety buffer. Minimizes total business cost (waste + 1.5x stockout).
- *waste_optimized* - Reduces buffer by 10% (buffer x 0.90). Accepts slightly more stockout risk in exchange for less over-preparation. Use for stores where food waste is the primary concern.
- *stockout_optimized* - Increases buffer by 6% (buffer x 1.06). Prepares slightly more to reduce the chance of running out. Use for high-traffic stores where stockouts damage customer retention.
- **API endpoints:** Table of all three endpoints with method, path, and description
- **Local development:** How to run with demo CSV data, how to connect to MySQL
- **Retraining guide:** Weekly retrain with decay weighting, 14-day forecast horizon cap
- **Project structure:** Directory layout with file descriptions
- **Expanding_mean leakage fix:** Summary of the Report B finding and how it was resolved

Summary of Deliverables

Task	Deliverable	Status
Expanding_mean leakage fix	evaluate.py, lag_features.py, feature_engineer.py	Complete
Modularized data_loader.py	src/forecast/data_loader.py	Complete
Modularized feature_engineer.py	src/forecast/feature_engineer.py	Complete
Modularized trainer.py	src/forecast/trainer.py	Complete
Modularized predictor.py	src/forecast/predictor.py	Complete
POST /ai/forecast/items endpoint	src/api/forecast_items_routes.py	Complete
README update	README.md	Complete

All SRS Section 8 success criteria addressed by my tasks are met. The expanding_mean leakage identified in Report B is fully resolved. The modularized codebase is production-ready and integrates cleanly with Yahya's data pipeline and endpoint implementations.